

Higher Order Programming

Samuel Ngugi Ndung'u
Student Number: 0542550

December 2018

1 Introduction

The following are the implementation details of the higher order programming project.

2 Triples

To add support for triples, we add new primitives to our metacircular evaluator. The functions availed to make, access and mutate triples are just regular scheme functions. To make sure they are usable in the meta-circular evaluator, we pass them to `prim/cc` which returns a lambda that receives two parameters: `continuation` and `arguments`. In the body of the returned lambda, we apply the primitive procedure ignoring the `continuation` parameter.

2.1 Making a triple

A triple is implemented as a list of four elements. The first element is symbol `triple` and the rest are the actual data elements held by the triple. The function `make-triple` creates and returns a new list representing a triple.

```
;; Make triple function  
(define (make-triple first second third)  
  (list 'triple first second third))
```

2.2 Retrieving triple elements

To facilitate retrieval of elements, we add the following scheme primitive procedures to our metacircular evaluator's list of primitives: `cadr`, `caddr` and `caddr`.

A function `first` returns the first data element of the triple which is actually the second element in the triple.

```
(define (first triple)  
  (cadr triple))
```

Second function gets the second data element of the triple which is at the third position of the triple data structure.

```
(define (second triple)
  (caddr triple))
```

third function gets the data element at position 4.

```
(define (third triple)
  (caddr triple))
```

2.3 Mutating triple elements

We will need the following scheme primitive procedures to achieve mutation of a triple: `set-car!` and `set-cdr!`. We therefore add them to the metacircular's list of primitives.

To change the first data element of the triple, `set-first!` takes the `cdr` of the triple and sets its `car` to the new value provided.

```
(define (set-first! triple new-value)
  (set-car! (cdr triple) new-value))
```

Mutating the second element, we take the `cddr` of the triple and set the `car` to the new value and returns symbol `done`.

```
(define (set-second! triple new-value)
  (set-car! (cddr triple) new-value)
  'done)
```

The third element of the triple is mutated by taking the `cdddr` of the triple and setting its `car` to the new value. We return symbol `done` to indicate success.

```
(define (set-third! triple new-value)
  (set-car! (cdddr triple) new-value)
  'done)
```

To test the functionalities mentioned above, use the sample program listed below.

```
(define (test x y z)
  (define triple
    (make-triple x y
      (make-triple z (* x 2) (* x 2))))
  ;; DISPLAY NEWLY CREATED TRIPLE
  (display triple)
  (newline)
  (newline)
  ;; MODIFY THE TRIPLE
  (set-first! triple (third triple))
  (set-second! triple (first triple)))
```

```

    (set-third! triple (second triple))
;; DISPLAY MODIFIED TRIPLE
    (display "==== Modified triple =====")
    (newline)
    (display triple)
    'ok)

```

Input the above program into the metacircular evaluator and then run (test 3 4 5). The output will be a new triple which is then modified and printed out.

3 Futures and Threads

3.1 Futures

A future object is represented by a list with the first element being the symbol `future-object`. The second element is a boolean value indicating whether the future is resolved or not. The third element is the expression which was marked by the future.

In the `evaluate`, if we encounter a an expression whose first symbol is future, then we create a future object for this expression using function `future`.

Every future object in the system is evaluated on it's own thread. In `eval-definition` function's continuation, we check if the value that is about to be bound to a variable in the environment is a future object. If so, we wrap the expression to be delayed in a procedure. The procedure holds the environment in which the future object will be evaluated. The body of the procedure is manipulated so that upon the evaluation of the future object, the future object is updated with the resolved value. See code snippet below.

```

(let* ((thread-body
      `(begin (set! ,(definition-variable exp)
                    (list 'future-object true ,(future-body value))))
      (proc (make-procedure '() thread-body env)))
      ...
    )

```

We use the newly constructed `proc` in the code above to create a thread in which the future object will be evaluated. `proc` is also used to replace the value stored in the future object. We now allow the future object to be bound to a variable. See the code below, for full details see functions `eval-definition` and `eval-def-future`.

```

(define-variable! (definition-variable exp)
  (list 'future-object #f proc)
  env)

```

The thread we create for the future object consists of the `proc`, environment and the continuation. The continuation is a lambda of one argument which removes the thread that just finished execution and returns the results. We queue the thread so that it can be executed at a later time. See code snippet below.

```
(queue-thread! (make-thread (procedure-body proc)
  (procedure-environment proc)
  (lambda(results)
    (remove-thread!)
    results))))
```

To `deref` a future object is to evaluate the body of the procedure it contains in the procedure environment and the a continuation which is a lambda of one argument that removes the thread for the just executed future object from the thread queue. The continuation also accesses the value of the resolved future and returns it.

```
(evaluate (procedure-body (future-body future))
  (procedure-environment (future-body future))
  (lambda(result)
    (remove-thread!)
    (future-body result))))
```

3.2 Threads

A thread is a list of three elements. It contains the expression to be evaluated, environment and continuation. To resume a thread is to evaluate the thread's expression in thread environment and with the thread continuation. To pause a thread is to take the current expression, environment and continuation as parameters to make a new thread which is and append it to the back of the thread queue. We then remove the active thread since it is now outdated. The newer version of the active thread is the one at the back of the queue.

A thread is allowed to invoke `evaluate` a fixed number of times. If it exceeds the preset value, it is switched and a new thread starts executing. Both the queue that keeps track of the threads and the counter that keeps track of how many times `evaluate` are defined in the global environment. Accessors and mutators of the threads queue and the counter are provided in the metacircular evaluator. See `check-execution` function to see how threads are switched.

4 Additional features

`switch` was implemented as an additional future. `switch` is a method that pauses the currently executing thread and starts the next thread.

Since our threads are stored in a global queue, if `switch` is encountered, it simply pauses the current thread, removes it from the front of the queue and starts executing the new active thread.

```

(define (switch)
  (cond ((not (empty-threads? (thread-queue)))
        (let ((active (active-thread)))
          (queue-thread! active)
          (remove-thread!)
          (let ((to-activate (active-thread)))
            (evaluate (thread-exp to-activate)
                      (thread-env to-activate)
                      (thread-cont to-activate)))))))

```

5 Test Program - Futures and Threads

Enter the program below in the metacircular evaluator input box.

```

(define (fib n)
  (if (= n 0)
      0
      (if (= n 1)
          1
          (begin
             (define f1 (future (fib (- n 1))))
             (define f2 (fib (- n 2)))
             (+ (deref f1) f2)))))

```

Once the evaluation of the above program finishes, enter

```

(define f (future (fib 5)))
;; Expected outputp 'ok

```

Now enter:

```

(fib 1) ;; Wait for it to evaluate
f ;; Wait for it to evaluate -> Expected output (unresolved future)
(fib 3) ;; Wait for it to evaluate
f ;; Wait for it to evaluate -> Expected output (resolved future)
(deref f) ;; To get the value of the future object.

```

6 Problems Encountered and Solutions

It was very problematic to determine how the future object value can be updated once the future object is resolved. The solution to this was to include code that updates the future object as it is being resolved.

The other problem was how to delay the evaluation of a future and ensure that the future is evaluated in the correct environment. The fact that many future objects can have the same name was an issue. The solution was to wrap the code of the future in a procedure that will keep the correct environment to evaluate the future object.